Section 0: Configuration

Setting	Value
Name	Ayushi Shringi
GitHub repository	https://github.com/ayushishringi/data260-0036
SID4	0036
PORT_BASE	8036 (8000 + 36 mod 900)
PREFIX	s0036
SEED	36
VERIFY_SEED	260036
DOMAIN_ID	4 (36 mod 8): open-source package vulnerability tracking
Primary entity	VulnerabilityReport (MySQL s0036_rel.vulnerability_reports)
Redis	Redis Stack 7.4.0 (container s0036-redis, port 6379), keys prefixed s0036:
Hardware	MacBook Pro, Apple M5 Pro, 24 GB
Local model	Ollama qwen3:4b (num_ctx 16384), no paid APIs
Branch	hw5
Tag	hw9
Commit	c6b1ee2 (HW9 code and verification)

The GitHub link was verified, and both collaborators (Sbnikitha, supriyaselvanganesan) have access.

Part 1: Redis View Counter

The primary entity is VulnerabilityReport (table s0036_rel.vulnerability_reports). increment_report_views(report_id) runs one Redis INCR on key s0036:views:report:<id> on every access. When count % 10 == 0 it updates view_count in MySQL and logs "□ Synced views to database", then returns the count. Only 1 in 10 views touches MySQL. After 25 views Redis holds 25 and MySQL holds 20, so up to 9 views exist only in Redis until the next sync.

```
((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 % make counter-hw09
.venv/bin/python3 scripts/run_hw9_counter.py
user=ayushishringi host=Ayushis-MacBook-Pro.local time=2026-11-05T17:49:01
report id=20074 (log4j-core, VULN-0036-HW9-01)  redis key=s0036:views:report:20074

=== Testing Counter ===

Access #1: Total = 1
Access #2: Total = 2
Access #3: Total = 3
Access #4: Total = 4
Access #5: Total = 5
Access #6: Total = 6
Access #7: Total = 7
Access #8: Total = 8
Access #9: Total = 9
Access #10: Total = 10
✓ Synced views to database
Access #11: Total = 11
Access #12: Total = 12
Access #13: Total = 13
Access #14: Total = 14
Access #15: Total = 15
Access #16: Total = 16
Access #17: Total = 17
Access #18: Total = 18
Access #19: Total = 19
Access #20: Total = 20
✓ Synced views to database
Access #21: Total = 21
Access #22: Total = 22
Access #23: Total = 23
Access #24: Total = 24
Access #25: Total = 25

Redis s0036:views:report:20074 = 25; MySQL vulnerability_reports.view_count = 20
```

Figure 1 – Part 1 counter output: 25 accesses, synced to MySQL at 10 and 20

```
((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 % docker exec s0036-redis redis-cli GET s0036:views:report:20074
mysql -uroot s0036_rel -e "SELECT id, report_code, package_name, view_count FROM vulnerability_reports WHERE id=20074"

25
+-------+-----------------+--------------+------------+
| id    | report_code     | package_name | view_count |
+-------+-----------------+--------------+------------+
| 20074 | VULN-0036-HW9-01 | log4j-core   |         20 |
+-------+-----------------+--------------+------------+
((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 %
```

Figure 2 – Redis count 25 vs MySQL view_count 20 for the same report

```python

class ReportCache:
    """Redis cache in front of the s0036_rel vulnerability_reports table."""

    def __init__(self, redis_client: redis.Redis | None = None, engine: Engine | None = None) -> None:
        if engine is None:
            from backend.database import engine as default_engine
            engine = default_engine
        self.redis_client = redis_client or get_redis()
        self.engine = engine

    def increment_report_views(self, report_id: int) -> int:
        """Count one view of a report in Redis; every 10 views persist the count to the database."""

        key = views_key(report_id)
        count = int(self.redis_client.incr(key))
        if count % SYNC_EVERY == 0:
            with self.engine.begin() as conn:
                result = conn.execute(text("UPDATE vulnerability_reports SET view_count = :count WHERE id = :id"),
                                      {"count": count, "id": report_id})
            if result.rowcount:
                logger.info("✓ Synced views to database")
            else:
                logger.warning("report %s not found; view count %s not synced", report_id, count)
        return count

```

Figure 3 – increment_report_views code (src/redis_store.py)

Part 2: Semantic Response Cache

#	Query type	is_cached	Similarity	Latency (ms)	Query
1	new	False	–	18,041.5	What is the severity of the Log4Shell vulnerability in log4j-core?
2	exact duplicate	True	1.0000	74.2	(same as 1)
3	paraphrase	True	0.8850	7.6	How severe is the log4j-core Log4Shell issue?
4	new	False	0.1636	55,115.2	Which lodash version fixes the template command injection?
5	paraphrase	True	0.9291	23.4	What lodash release patches the command injection in template?
6	new	False	0.1568	85,572.6	How do I generate an SBOM for a Python project?
7	exact duplicate	True	1.0000	15.0	(same as 4)
8	new	False	0.2083	18,200.2	Which requests version stops leaking the Proxy-Authorization header?
9	paraphrase	True	0.9490	17.3	What version of requests fixes the Proxy-Authorization header leak?
10	new	False	0.1990	9,034.1	What does a CWE identifier represent?
11	exact duplicate	True	1.0000	7.6	(same as 6)
12	paraphrase	False	0.8366	8,858.7	What is a CWE ID used for?

The cache sits at the ingress, before s0036_inbox. Each query is embedded with all-MiniLM-L6-v2 (384-dim, normalized), and a RediSearch KNN-1 search on index s0036:idx:qcache returns the nearest cached query. Similarity is 1 − cosine distance. If similarity ≥ 0.85 (and the data version still matches, see 3.5), the stored final answer is returned with is_cached=true, without running the agents. Otherwise the question goes through the HW8 Planner → Writer → Reviewer pipeline over Kafka. The approved answer is then stored with its query, embedding, data-version marker and record ids, and returned with is_cached=false. Each step is timed (embed_ms, search_ms, pipeline_ms, store_ms).

Measured: hit rate 6/12 = 50%. Cached answers averaged **24.2 ms**; non-cached ones **32,470.4 ms** (~32.5 s), nearly all of it the two LLM calls. Exact duplicates hit at similarity 1.0, and three paraphrases hit at 0.885–0.949. "What is a CWE ID used for?" scored 0.8366 and missed: a real paraphrase rejected by 0.85, the first sign that the threshold matters.



Figure 4 – Part 2 demo: the 12 queries with CACHE HIT / CACHE MISS logs, hit rate and average latencies

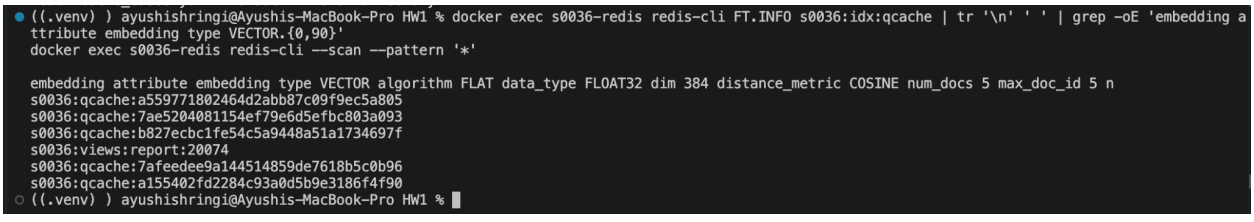


Figure 5 – Vector index s0036:idx:qcache (FLAT, FLOAT32, DIM 384, COSINE) and the s0036 cache keys

```
    def ensure_index(self) -> None:
        """Create the FLAT (exact) cosine vector index over
        s0036:qcache:* hashes if it does not exist."""

        try:
            self.r.ft(self.index_name).info()
        except redis.ResponseError:
            self.r.ft(self.index_name).create_index(
                [TextField("query"), TextField("answer"), TagField
                ("data_version"), TagField("record_ids"),
                 NumericField("created_at"),
                 VectorField("embedding", "FLAT", {"TYPE": "FLOAT32",
                 "DIM": DIM, "DISTANCE_METRIC": "COSINE"})],
                definition=IndexDefinition(prefix=[self.key_prefix],
                index_type=IndexType.HASH))

    def clear(self) -> int:
        """Delete every cache entry (and keep the index)."""

        keys = list(self.r.scan_iter(match=f"{self.key_prefix}*",
        count=500))
        if keys:
            self.r.delete(*keys)
        return len(keys)

    def size(self) -> int:
        return int(self.r.ft(self.index_name).info()["num_docs"])
```

Figure 6 – ensure_index: creating the vector index (src/semantic_cache.py)

```
        vector = self.embedder.embed(query)
        t1 = time.perf_counter()
        timings["embed_ms"] = ms(t0, t1)
        best = self.cache.nearest(vector)
        t2 = time.perf_counter()
        timings["search_ms"] = ms(t1, t2)
        similarity = best["similarity"] if best else None
        result: dict[str, Any] = {"query": query, "threshold": threshold, "similarity": similari
            ty,
                                  "matched_query": best["query"] if best else None}

        stale = False
        if best and similarity >= threshold:
            if self.invalidation == "version":
                current = advisory_lookup.current_version(best["record_ids"])
                timings["version_check_ms"] = ms(t2, time.perf_counter())
                stale = current != best["data_version"]
            if stale:
                self.cache.delete(best["key"])
                log(self.component, "STALE entry invalidated (data_version changed)", similarity
                    =similarity,
                    cached_query=best["query"], stored_version=best["data_version"], current_ver
                        sion=current)
            else:
                latency = ms(t0, time.perf_counter())
                log(self.component, "CACHE HIT", similarity=similarity, threshold=threshold,
                    cached_query=best["query"], latency_ms=latency)
                return {**result, "is_cached": True, "answer": best["answer"], "data_version": b
                    est["data_version"],
                        "record_ids": best["record_ids"], "latency_ms": latency, "timings": timi
                            ngs}
```

```
        # The data version is read BEFORE the agents run, so an update during the run makes the
            entry stale.
        records = advisory_lookup.find_records(query)
        version = advisory_lookup.data_version(records)
        t3 = time.perf_counter()
        final = self.pipeline.ask(query)
        t4 = time.perf_counter()
        timings["pipeline_ms"] = ms(t3, t4)
        answer, status = final.get("answer", ""), final.get("status")
        stored_key = None
        if self.store_on_miss and status == "approved":
            stored_key = self.cache.store(query, vector, answer, version, [r["id"] for r in reco
                rds],
```

Figure 7 – CachedQA.ask (src/semantic_cache.py): embed, KNN search, threshold and version check, cache hit; on a miss, the agent pipeline and store

Part 3: Threshold Economics and Stale Answers

3.1 Labeled test set

The test set has 10 seed questions that are cached first, plus 40 labeled cases: 10 exact duplicates of the seeds (should hit), 10 paraphrases with the same package and question (should hit), 10 superficially similar questions with a different correct answer (should miss), and 10 unrelated domain questions (should miss). The similar-but-different cases change the package (lodash → pillow), the field asked (severity → CVE), or the end of the version range ("first introduced" vs "fixes").

Labeling rule: a hit is correct only if the main answer text answers the question that was asked (the right package and the right field). It is not enough for the fact to appear somewhere in the text. The effect of this rule is checked in 3.4.

Committed before the sweep: cache_cases.yaml was committed in de51eaa (2026-11-05) and never edited. The sweep reported here ran on 2026-11-08. Its log prints "last commit: de51eaa" before any result (Figure 9), and sweep_results.json records the same commit and sha256 (eb413fcb8db3f094…). Figure 10 shows that the working file, the committed file and the sweep's recorded hash all match. make verify-hw09 re-checks this. The full file is in Appendix A.

Category	Count	Expected	Example
Exact duplicate	10	hit	"Which lodash version fixes the template command injection?"
Paraphrase	10	hit	"What lodash release patches the command injection in template?"
Superficially similar	10	miss	"Which requests version first introduced the Proxy-Authorization header leak?"
Unrelated	10	miss	"How do I generate an SBOM for a Python project?"

```
((.venv) ) ayushishringi@Ayushis-MacBook-Pro HW1 % cd "/Users/ayushishringi/Desktop/DATA 260/HW1/data260-0036"
git log --oneline -- cache_cases.yaml
grep -c "category:" cache_cases.yaml
head -30 cache_cases.yaml

de51eaa HW9: labeled semantic-cache test set (cache_cases.yaml)
40
# HW9 Part 3.1 - labeled semantic-cache test set (s0036, DOMAIN_ID 4: open-source package vulnerabilities).
#
# Labels were written BEFORE any threshold sweep and must not be edited after seeing results.
# Labeling mistakes found later are recorded in reports/hw09/LABEL_NOTES.md instead.
#
# How the set is used (scripts/run_hw9_sweep.py):
#   1. The cache is cleared and the 10 seed_queries are answered by the HW8 agents and cached.
#   2. For each threshold, the 40 cases are asked against that same seeded cache
#      (store_on_miss=false, so every threshold sees an identical cache).
#   expected: hit  -> the correct answer is the cached answer of seed `target`
#   expected: miss -> no cached answer is correct (`near` = the seed it superficially resembles)
#   A false hit = a cache hit whose cached answer is not correct for the incoming query.

seed_queries:
  - {id: S01, query: "What is the severity of the Log4Shell vulnerability in log4j-core?"}
  - {id: S02, query: "Which lodash version fixes the template command injection?"}
  - {id: S03, query: "What versions of minimist are affected by the prototype pollution bug?"}
  - {id: S04, query: "How should I fix the PyYAML full_load code execution vulnerability?"}
  - {id: S05, query: "Which requests version stops leaking the Proxy-Authorization header?"}
  - {id: S06, query: "Is Spring4Shell in spring-beans a critical vulnerability?"}
  - {id: S07, query: "What is the CVE for the axios XSRF-TOKEN leak?"}
  - {id: S08, query: "Which jackson-databind version fixes the deeply nested JSON denial of service?"}
  - {id: S09, query: "What is the impact of the node-fetch cookie leak on redirect?"}
  - {id: S10, query: "Which ws version fixes the header-count denial of service?"}

cases:
  # ---- 10 exact duplicates (should hit) ----
  - {id: C01, category: exact_duplicate, expected: hit, target: S01, query: "What is the severity of the Log4Shell vulnerability in log4j-core?"}
  - {id: C02, category: exact_duplicate, expected: hit, target: S02, query: "Which lodash version fixes the template command injection?"}
  - {id: C03, category: exact_duplicate, expected: hit, target: S03, query: "What versions of minimist are affected by the prototype pollution bug?
"}
((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 %
```

Figure 8 – cache_cases.yaml: seed queries and the labeled cases

3.2 Threshold sweep

Threshold	Hit rate	False-hit rate	Mean latency (ms)	Total cost
0.75	65.0% (26/40)	15.0% (6/40)	14,826.2	$90.14
0.80	62.5% (25/40)	12.5% (5/40)	13,290.6	$75.15
0.85	57.5% (23/40)	10.0% (4/40)	14,364.6	$60.17
0.90	50.0% (20/40)	10.0% (4/40)	17,894.3	$60.20
0.95	27.5% (11/40)	0.0% (0/40)	25,299.4	$0.29

Threshold	Correct hits	False hits	Missed hits	Correct misses
0.75	20	6	0	14
0.80	20	5	0	15
0.85	19	4	1	16
0.90	16	4	4	16
0.95	11	0	9	20

Every threshold was run against the same 10-entry seeded cache (store_on_miss=False), so the results are comparable. Every miss ran the real Planner → Writer → Reviewer pipeline. Hit rate = hits / 40. False-hit rate = hits whose cached answer is wrong for the query, divided by 40. Mean latency is averaged over all 40 queries: hits take about 6–12 ms and misses

about 34–42 s on average. The 0.75 run was the first and its misses were slower (42.3 s each), so its mean latency is slightly above that of 0.80 even though it has more hits.

Lowering the threshold raises the hit rate, but false hits rise from 0 to 6. Four of the similar-but-different questions score 0.90–0.93, higher than several real paraphrases (0.83–0.89), so no single threshold separates the two groups. 0.95 removes every false hit but also turns away 9 of the 10 paraphrases.

```
(.venv) ayushishringi@Ayushis-MacBook-Pro data260-0036 % echo "Name: Ayushi Shringi | SID4 0036"
Name: Ayushi Shringi | SID4 0036
(.venv) ayushishringi@Ayushis-MacBook-Pro data260-0036 % grep -E "user=|cache_cases|^=== Threshold|hit rate" rep
orts/hw09/raw/sweep_console.log
user=ayushishringi host=Ayushis-MacBook-Pro.local time=2026-11-08T16:09:33
cache_cases.yaml sha256=eb413fcb8db3f094 last commit: de51eaa 2026-11-05T09:03:55-08:00
=== Threshold 0.75 ===
  -> hit rate 65.0%, false-hit rate 15.0%, mean latency 14826 ms
=== Threshold 0.80 ===
  -> hit rate 62.5%, false-hit rate 12.5%, mean latency 13291 ms
=== Threshold 0.85 ===
  -> hit rate 57.5%, false-hit rate 10.0%, mean latency 14365 ms
=== Threshold 0.90 ===
  -> hit rate 50.0%, false-hit rate 10.0%, mean latency 17894 ms
=== Threshold 0.95 ===
  -> hit rate 27.5%, false-hit rate 0.0%, mean latency 25299 ms
(.venv) ayushishringi@Ayushis-MacBook-Pro data260-0036 % grep -o '"cases_commit": "[^"]*"' reports/hw09/raw/swee
p_results.json
"cases_commit": "de51eaa 2026-11-05T09:03:55-08:00"
(.venv) ayushishringi@Ayushis-MacBook-Pro data260-0036 %
```

Figure 9 – Threshold sweep log: the label file's commit (de51eaa) is printed before any result, then hit rate, false-hit rate and mean latency per threshold

```
(.venv) ayushishringi@Ayushis-MacBook-Pro data260-0036 % echo "Name: Ayushi Shringi | SID4 0036"
Name: Ayushi Shringi | SID4 0036
(.venv) ayushishringi@Ayushis-MacBook-Pro data260-0036 % git log --oneline -1 de51eaa
de51eaa HW9: labeled semantic-cache test set (cache_cases.yaml)
(.venv) ayushishringi@Ayushis-MacBook-Pro data260-0036 % shasum -a 256 cache_cases.yaml
eb413fcb8db3f09449c8031ada4649202dcfb91b2f2240bb4a5a24a872603ee8  cache_cases.yaml
(.venv) ayushishringi@Ayushis-MacBook-Pro data260-0036 % git show de51eaa:cache_cases.yaml | shasum -a 256
eb413fcb8db3f09449c8031ada4649202dcfb91b2f2240bb4a5a24a872603ee8  -
(.venv) ayushishringi@Ayushis-MacBook-Pro data260-0036 % grep -o '"cases_sha256": "[0-9a-f]*"' reports/hw09/raw/sweep_results.json
"cases_sha256": "eb413fcb8db3f09449c8031ada4649202dcfb91b2f2240bb4a5a24a872603ee8"
(.venv) ayushishringi@Ayushis-MacBook-Pro data260-0036 %
```

Figure 10 – sha256 of cache_cases.yaml: working file = committed file (de51eaa) = hash recorded by the sweep

3.3 One false hit in full

Incoming query (C25, labeled should-miss): Which requests version first introduced the Proxy-Authorization header leak?

Cached query (S05): Which requests version stops leaking the Proxy-Authorization header?

Similarity score: 0.9267. That is above the 0.85 threshold, so the cached answer was returned without running the agents.

Cached answer returned (excerpt): "…The fix was implemented in version 2.31.0 … Users must upgrade to requests 2.31.0+ to prevent the Proxy-Authorization leak." Source record: VULN-0036-HW9-05: requests affected >= 2.3.0, < 2.31.0, severity medium,

CVE-2023-32681, fixed in 2.31.0.

Correct answer (from the agents when the same query missed at threshold 0.95, excerpt): "The vulnerability CVE-2023-32681 … was first introduced in requests version 2.3.0 … The vulnerability was fixed in version 2.31.0."

Why it is wrong: the user asked when the bug started (2.3.0). The cached answer tells them how to fix it (2.31.0) and never answers the question asked. The source line does show the range ">= 2.3.0", but under the labeling rule in 3.1 the answer has to address the question asked, so it is a false hit (see LABEL_NOTES.md).

Likely embedding confusion: the two queries share every distinctive word: "requests", "Proxy-Authorization", "header", "leak", "version". The sentence embedding (all-MiniLM-L6-v2) averages over the words, so these shared topic words dominate the vector. The only difference is the intent: "first introduced" asks for the start of the affected range, and "stops leaking" asks for the end. The embedding captures what the question is about, not which direction it asks, so the two land very close together (0.93).

All false hits in the sweep: C25 (0.9267), C30 (0.9245), C26 (0.9106), C24 (0.9000), C21 (0.8333), C29 (0.7970). Every one is a "same topic, different question" case, which is why only a very high threshold (0.95) removes them.

3.4 Cost model and threshold choice

Cache-miss cost: C_miss = $0.0100 per miss. A miss runs the full agent pipeline: the Planner and Writer each call qwen3:4b, and the Reviewer checks the draft. Across the 95 misses in the sweep, the measured mean pipeline time was 36.1 s of GPU time. Priced at $1.00/hour for a comparable single cloud GPU, that is 36.1 / 3600 × $1.00 = $0.0100. The user also waits about 36 s, which this figure doesn't include.

False-hit / support cost: C_false = $15.00 per false hit. A false hit returns a wrong security fact, such as the wrong CVE, severity or version, with no warning. A developer acts on it or files a ticket. We assume 15 minutes of engineer time at $60/hour to notice, triage and correct it: 0.25 h × $60 = $15. This is a minimum: if someone keeps a still-vulnerable version because of a wrong "fixed in" answer, the real cost is far higher.

Ratio: C_false / C_miss ≈ 1,498, so one false hit costs as much as about 1,500 agent runs. A correct hit costs almost nothing (a few ms of CPU).

Formula: Total cost = misses × C_miss + false hits × C_false

Threshold	Misses	False hits	Miss cost	False-hit cost	Total cost
0.75	14	6	$0.14	$90.00	$90.14
0.80	15	5	$0.15	$75.00	$75.15
0.85	17	4	$0.17	$60.00	$60.17
0.90	20	4	$0.20	$60.00	$60.20
0.95 (ship)	**29**	**0**	**$0.29**	**$0.00**	**$0.29**

Threshold to ship: 0.95. It has the lowest total cost ($0.29) because it is the only threshold with zero false hits. Every lower threshold saves only a few cents in misses but adds $15 for every wrong answer. For a security domain, where a wrong version or severity can leave a system exposed, correctness matters more than speed.

Sensitivity to the support cost: 0.85 would only beat 0.95 if a false hit cost less than 3 misses (about $0.03), meaning a wrong vulnerability answer would have to be almost free. So the choice of 0.95 holds for any realistic support cost.

What 0.95 gives up: its 11 hits are the 10 exact duplicates plus a single paraphrase (C16, 0.9698). 9 of the 10 paraphrases go to the agents, and mean latency is about 25.3 s. In practice, 0.95 turns the semantic cache into an exact-match cache. A plain threshold cannot do better here, because the dangerous near-misses (0.90–0.93) score higher than most real paraphrases. Three changes would win back the paraphrase hits without the false hits:

- **Field-aware cache key:** extract the package and the field asked (fixed version, first affected version, severity, CVE, impact) from the query, store them with the entry, and serve a hit only when both match. That alone blocks all six false hits, which all differ in package or field, while every paraphrase still matches.
- **Rerank in the grey zone:** serve similarity ≥ 0.95 directly. For 0.85–0.95, re-score the pair with a cross-encoder, which reads both questions together and catches "first introduced" vs "fixes", before accepting the hit.
- **Answer-level check:** before returning a grey-zone hit, check that the cached answer contains a value of the requested field type for that package. This costs a few ms, compared with about 36 s for a miss.

Sensitivity to the labels: the six false hits are arguable. The Writer appends the full source record to every answer, so each of them contains the needed fact somewhere (for example ">= 2.3.0" in C25). Under a looser rule ("the fact appears anywhere in the text") there would be no false hits at any threshold, and 0.75 would be cheapest ($0.14). The strict rule is used because a user reads the answer, not the appended record line. C25's answer explicitly tells the user to upgrade to 2.31.0 when they asked which version introduced the bug. So the ship decision depends on how strictly "correct" is defined. A field-aware key removes that dependence, because the arguable cases are exactly the ones whose field differs.

```python
# ---- Part 3.4 cost model
GPU_DOLLARS_PER_HOUR = 1.00  # price of a comparable single cloud GPU (T4/L4 class) running the local model
SUPPORT_MINUTES = 15  # engineer time to notice, triage and correct one wrong security answer
ENGINEER_DOLLARS_PER_HOUR = 60.0
C_FALSE_HIT = round(SUPPORT_MINUTES / 60 * ENGINEER_DOLLARS_PER_HOUR, 2)

def cost_model(sweep: dict) -> dict:
    miss_ms = [r["timings"]["pipeline_ms"] for t in sweep["thresholds"].values()
    for r in t["rows"]
                if not r["is_cached"]]
    mean_miss_s = statistics.mean(miss_ms) / 1000
    c_miss = mean_miss_s / 3600 * GPU_DOLLARS_PER_HOUR
    totals = {}
    for t, block in sweep["thresholds"].items():
        s = block["summary"]
        totals[t] = {"misses": s["misses"], "false_hits": s["false_hit"],
                     "miss_cost": s["misses"] * c_miss, "false_cost": s
                     ["false_hit"] * C_FALSE_HIT}
        totals[t]["total"] = totals[t]["miss_cost"] + totals[t]["false_cost"]
    best = min(totals, key=lambda t: (round(totals[t]["total"], 6), -sweep
    ["thresholds"][t]["summary"]["hit_rate"]))
    return {"mean_miss_s": mean_miss_s, "c_miss": c_miss, "totals": totals,
    "best": best, "n_misses": len(miss_ms)}
```

Figure 11 – Cost constants and cost_model (scripts/build_hw9_metrics.py)

3.5 Stale answers and invalidation

1. Cached query and record. The query "What is the severity of the requests Proxy-Authorization header leak?" depends on the s0036_rel.vulnerability_reports row VULN-0036-HW9-05 (id 20078, package requests, severity medium). The Writer reads this row and cites it in the answer. When the answer is cached, its data-version marker v-69591878aff271b6 and the record id are stored with it.

2. Update. UPDATE vulnerability_reports SET severity='high' WHERE report_code='VULN-0036-HW9-05'. This is a simulated re-scoring of the advisory, and the row was restored to medium after the demo. The record's marker changed to v-4bdee5bf2753d7c2.

3. Stale hit. With no check (invalidation=none), the same query hit the cache in 5.7 ms and returned the old answer, which says severity medium while the database says high.

4. Invalidation strategy: version check on read. Each cache entry stores a data_version, a hash of the content and updated_at of the records the answer used, plus their ids. On every hit, the cache re-reads those rows by primary key and recomputes the marker. If it differs, the entry is deleted and the query is treated as a miss. In the demo the cache logged "STALE entry invalidated", the agents reran (11.7 s) and answered "high", and the new answer was cached under the new marker. The next ask was a fresh hit in 5.5 ms.

Step	is_cached	Similarity	Latency (ms)	Answer says	Database says	Result
1. First ask	False	–	14,990.7	medium	medium	fresh
2. Paraphrase before update	True	0.9079	27.0	medium	medium	fresh
3. After update, no check	True	1.0000	5.7	medium	high	**STALE**
4. After update, version check	False	1.0000	11,673.5	high	high	fresh
5. Ask again	True	1.0000	5.5	high	high	fresh

5. Trade-off. No tracked record is ever served stale, and nothing that writes to the database has to remember to clear the cache. The cost:

- Every hit makes one extra MySQL primary-key read (a few ms), so hits are slightly slower.
- The cache now depends on MySQL being up.
- It only tracks the records found in the query. If a new advisory row is added that should change an answer, it isn't noticed until the entry is replaced.
- A TTL (time-to-live) would be cheaper on every hit but would serve stale answers until the entry expires.

```
(.venv) ayushishringi@Ayushis-MacBook-Pro data260-0036 % echo "Name: Ayushi Shringi | SID4 0036"
Name: Ayushi Shringi | SID4 0036
(.venv) ayushishringi@Ayushis-MacBook-Pro data260-0036 % grep -v -e "Loading weights" -e "^$" reports/hw09/raw/s
tale_console.log
.venv/bin/python3 -u scripts/run_hw9_stale.py
user=ayushishringi host=Ayushis-MacBook-Pro.local time=2026-11-08T17:11:10
column vulnerability_reports.view_count already exists
HW9 advisory rows: inserted=0 restored=0 unchanged=23
record VULN-0036-HW9-05 before: package=requests severity=medium updated_at=2026-11-06 15:39:37 data_version=v-6
9591878aff271b6
2026-11-08T17:11:14.481-08:00 [stale-demo] CACHE MISS best_similarity=null threshold=0.85 query="What is the sev
erity of the requests Proxy-Authorization header leak?" reason="empty cache"
2026-11-08T17:11:29.460-08:00 [stale-demo] ANSWERED by agent pipeline correlation_id="f818d627-b988-477a-b32d-4b
244aa5f1d9" status="approved" stored=true latency_ms=14990.71
1. first ask (miss -> agents -> cached)
   is_cached=False similarity=- latency_ms=14990.7 data_version=v-69591878aff271b6
   answer says severity=medium; database says severity=medium -> fresh
2026-11-08T17:11:29.489-08:00 [stale-demo] CACHE HIT similarity=0.9079 threshold=0.85 cached_query="What is the
severity of the requests Proxy-Authorization header leak?" latency_ms=27.04
2. paraphrase before the update (hit)
   is_cached=True similarity=0.9079 latency_ms=27.0 data_version=v-69591878aff271b6
   answer says severity=medium; database says severity=medium -> fresh
UPDATE vulnerability_reports SET severity='high' WHERE report_code='VULN-0036-HW9-05'
record after: severity=high updated_at=2026-11-08 17:11:29 data_version=v-4bdee5bf2753d7c2
2026-11-08T17:11:29.513-08:00 [stale-demo] CACHE HIT similarity=1.0 threshold=0.85 cached_query="What is the sev
erity of the requests Proxy-Authorization header leak?" latency_ms=5.66
3. invalidation=none -> STALE HIT (old answer returned)
   is_cached=True similarity=1.0000 latency_ms=5.7 data_version=v-69591878aff271b6
   answer says severity=medium; database says severity=high -> STALE
2026-11-08T17:11:29.520-08:00 [stale-demo] STALE entry invalidated (data_version changed) similarity=1.0 cached_
query="What is the severity of the requests Proxy-Authorization header leak?" stored_version="v-69591878aff271b6
" current_version="v-4bdee5bf2753d7c2"
2026-11-08T17:11:29.520-08:00 [stale-demo] CACHE MISS best_similarity=1.0 threshold=0.85 query="What is the seve
rity of the requests Proxy-Authorization header leak?" reason="stale"
2026-11-08T17:11:41.187-08:00 [stale-demo] ANSWERED by agent pipeline correlation_id="574cf7c3-2e22-4241-a3aa-7b
c2de893b3d" status="approved" stored=true latency_ms=11673.48
4. invalidation=version -> marker mismatch, entry deleted, agents rerun
   is_cached=False similarity=1.0000 latency_ms=11673.5 data_version=v-4bdee5bf2753d7c2
   answer says severity=high; database says severity=high -> fresh
2026-11-08T17:11:41.193-08:00 [stale-demo] CACHE HIT similarity=1.0 threshold=0.85 cached_query="What is the sev
erity of the requests Proxy-Authorization header leak?" latency_ms=5.53
5. invalidation=version, ask again -> fresh hit
   is_cached=True similarity=1.0000 latency_ms=5.5 data_version=v-4bdee5bf2753d7c2
   answer says severity=high; database says severity=high -> fresh
restoring the record to the reference value
column vulnerability_reports.view_count already exists
HW9 advisory rows: inserted=0 restored=1 unchanged=22
(.venv) ayushishringi@Ayushis-MacBook-Pro data260-0036 %
```

Figure 12 – Stale-answer demo (make stale-hw09): cached answer, UPDATE, stale hit with no check, invalidation with the version check, fresh hit

```
57  def data_version(records: list[dict[str, Any]]) -> str:
58      """Data-version marker: hash of the content and updated_at of the records an answer used."""
59
60      if not records:
61          return "none"
62      parts = [f"{r['id']}|{r['affected_version']}|{r['severity']}|{r['description']}|{r['updated_
            at']}"
63               for r in sorted(records, key=lambda r: r["id"])]
64      return "v-" + hashlib.sha1("\n".join(parts).encode()).hexdigest()[:16]
65
66
67  def current_version(record_ids: list[int]) -> str:
68      """Data-version marker computed from the rows as they are in the database right now."""
```

Figure 13 – data_version marker (src/advisory_lookup.py); the check on read is lines 148–158 of Figure 7

Part 4: Self-Improving Multi-Agent System

Datasets

data/hw9/reference_facts.json is the ground truth: 23 packages, each with its CVE, severity, affected version range and first fixed version, compiled from public advisories. The same rows are in s0036_rel.vulnerability_reports. The input files hold only an id and a question, never the answer:

- training_data.json: 10 questions (Appendix D)
- validation_data.json: 10 questions (Appendix B)
- test_data.json: 12 questions (Appendix C)

Each file includes questions about packages with no record (for example "fastjsonx"), and the correct behavior is to decline. The test file was used only once, in the final run.

4.1 Repair with evaluator feedback

The Answering Agent (qwen3:4b) answers. The Evaluator Agent finds the package in the question, works out which fields are being asked (fixed version, severity, CVE, affected range), and checks them against the reference. It returns PASS or REVISE, a short explanation and correction feedback. On REVISE, the Answering Agent gets its previous answer plus the feedback and writes a revised answer, which is evaluated again. The controlled case forces an incomplete first answer for "Which version of lodash fixes CVE-2021-23337?" ("…upgrade to a newer release"):

1. The evaluator returns REVISE: "First fixed version is missing", with feedback giving 4.17.21.
2. The revised answer states 4.17.21 and gets PASS.
3. 2 attempts in total.

Complete revision sequence (controlled_01, from part4_controlled.json):

Step	Content
Question	Which version of lodash fixes CVE-2021-23337?
Initial answer (attempt 1)	“There is a known security issue in lodash; you should upgrade to a newer release.” (forced incomplete)
Evaluation 1	REVISE: “First fixed version is missing.”
Evaluator feedback	“The reference record for lodash gives the first fixed version as 4.17.21; state it exactly.”
Revised answer (attempt 2)	“The first version of lodash that fixes CVE-2021-23337 is 4.17.21.”
Final evaluation	PASS: “Matches the reference for lodash on: first fixed version.”
Number of attempts	2 (the maximum)
Final response	“The first version of lodash that fixes CVE-2021-23337 is 4.17.21.” (PASS)

4.2 Lessons

The Evaluator counts failure patterns in initial answers. When the same pattern appears twice, it saves a lesson to lessons.json with lesson_id, failure_pattern, correction, created_at and use_count. In training:

- lesson_001 (wrong fixed version) was created after train_05, the second fixed-version mistake.
- lesson_002 (wrong severity) was created after train_06.

Before answering, the Answering Agent loads the lessons that match the question's field, adds their corrections to its prompt and follows the action, which is to look up the package's record in s0036_rel. Each use increments use_count. Two examples of reuse:

- In training, train_08 loaded lesson_001 and passed on its first attempt (Figure 14).
- A later question, "Which version of ws fixes the header-count denial of service?", loaded lesson_001 (use_count 2 → 3), looked up the record, and answered 8.17.1 with PASS in 1 attempt (Figure 17).

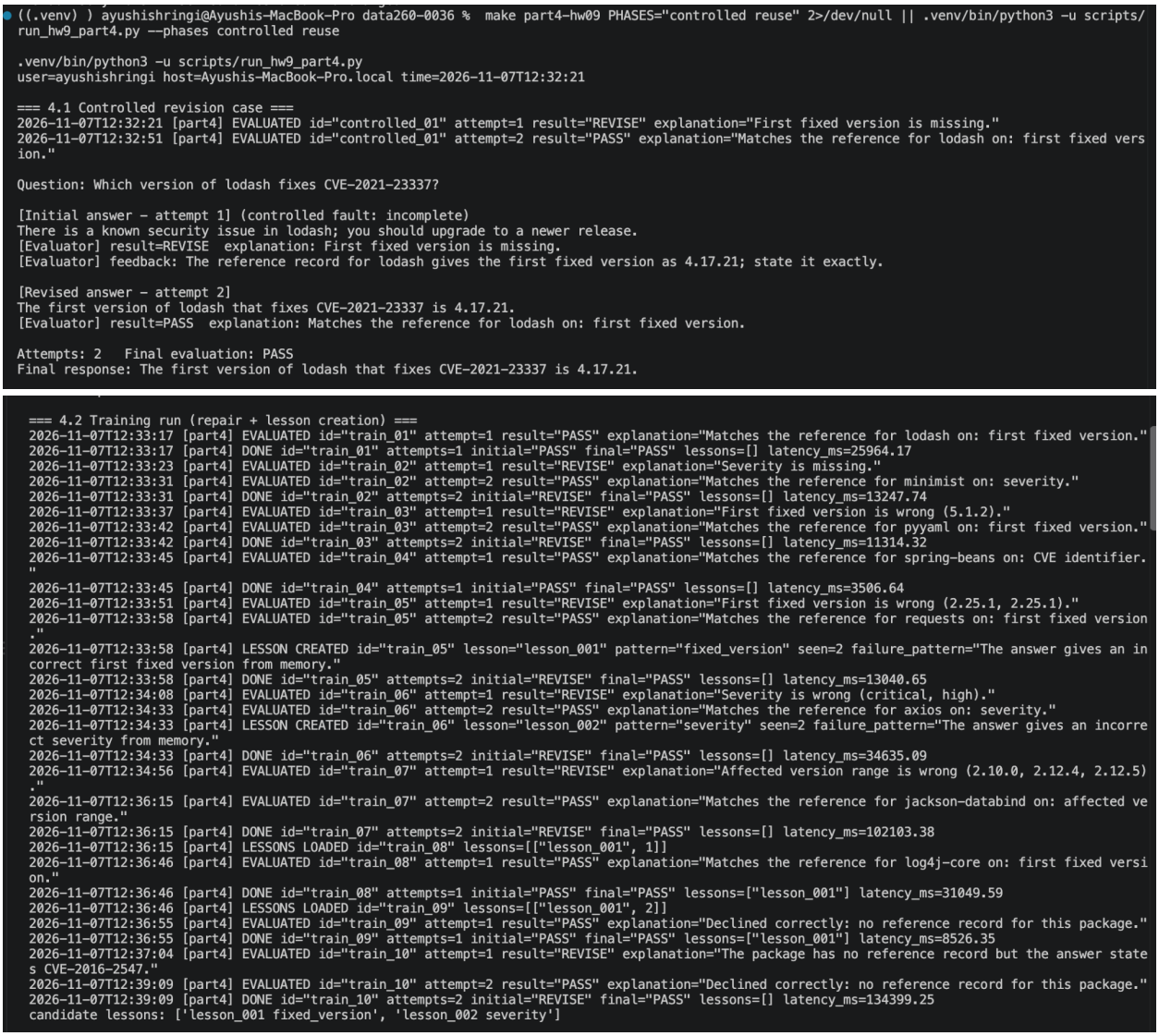


Figure 14 – Part 4 run (make part4-hw09): 4.1 controlled case (REVISE with feedback, then PASS on attempt 2), then 4.2 training with LESSON CREATED for lesson_001 and lesson_002 and train_08 reusing lesson_001

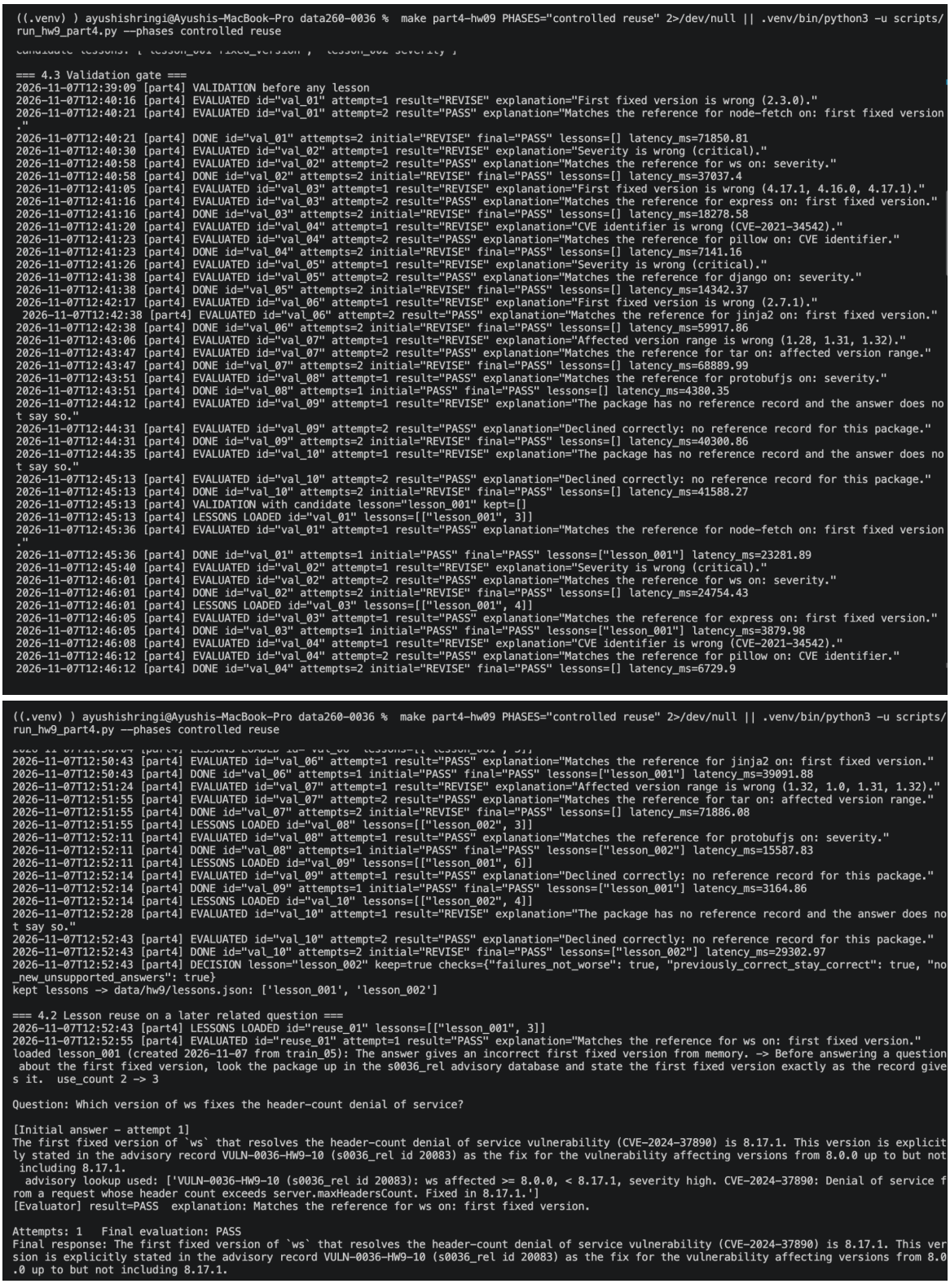
4.3 Validation

Each candidate lesson was run on validation_data.json before and after adding it. It is kept only if all three hold:

1. Failures do not increase.
2. Every previously correct answer stays correct.
3. No unknown-package question gets an unsupported answer.

Step	Initial answer correct	Final correct	Avg attempts	Kept
No lessons	1/10	10/10	1.9	–
+ lesson_001	5/10	10/10	1.5	yes
+ lesson_002	7/10	10/10	1.3	yes

Both lessons passed all three checks, so lessons.json keeps both and none was rejected (Figure 15). Figure 16 shows the code for the three keep rules.



```
((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 %  make part4-hw09 PHASES="controlled reuse" 2>/dev/null || .venv/bin/python3 -u scripts/
run_hw9_part4.py --phases controlled reuse

=== 4.3 Validation gate ===
2026-11-07T12:39:09 [part4] VALIDATION before any lesson
2026-11-07T12:40:16 [part4] EVALUATED id="val_01" attempt=1 result="REVISE" explanation="First fixed version is wrong (2.3.0)."
2026-11-07T12:40:21 [part4] EVALUATED id="val_01" attempt=2 result="PASS" explanation="Matches the reference for node-fetch on: first fixed version
."
2026-11-07T12:40:21 [part4] DONE id="val_01" attempts=2 initial="REVISE" final="PASS" lessons=[] latency_ms=71850.81
2026-11-07T12:40:30 [part4] EVALUATED id="val_02" attempt=1 result="REVISE" explanation="Severity is wrong (critical)."
2026-11-07T12:40:58 [part4] EVALUATED id="val_02" attempt=2 result="PASS" explanation="Matches the reference for ws on: severity."
2026-11-07T12:40:58 [part4] DONE id="val_02" attempts=2 initial="REVISE" final="PASS" lessons=[] latency_ms=37037.4
2026-11-07T12:41:05 [part4] EVALUATED id="val_03" attempt=1 result="REVISE" explanation="First fixed version is wrong (4.17.1, 4.16.0, 4.17.1)."
2026-11-07T12:41:16 [part4] EVALUATED id="val_03" attempt=2 result="PASS" explanation="Matches the reference for express on: first fixed version."
2026-11-07T12:41:16 [part4] DONE id="val_03" attempts=2 initial="REVISE" final="PASS" lessons=[] latency_ms=18278.58
2026-11-07T12:41:20 [part4] EVALUATED id="val_04" attempt=1 result="REVISE" explanation="CVE identifier is wrong (CVE-2021-34542)."
2026-11-07T12:41:23 [part4] EVALUATED id="val_04" attempt=2 result="PASS" explanation="Matches the reference for pillow on: CVE identifier."
2026-11-07T12:41:23 [part4] DONE id="val_04" attempts=2 initial="REVISE" final="PASS" lessons=[] latency_ms=7141.16
2026-11-07T12:41:26 [part4] EVALUATED id="val_05" attempt=1 result="REVISE" explanation="Severity is wrong (critical)."
2026-11-07T12:41:38 [part4] EVALUATED id="val_05" attempt=2 result="PASS" explanation="Matches the reference for django on: severity."
2026-11-07T12:41:38 [part4] DONE id="val_05" attempts=2 initial="REVISE" final="PASS" lessons=[] latency_ms=14342.37
2026-11-07T12:42:17 [part4] EVALUATED id="val_06" attempt=1 result="REVISE" explanation="First fixed version is wrong (2.7.1)."
 2026-11-07T12:42:38 [part4] EVALUATED id="val_06" attempt=2 result="PASS" explanation="Matches the reference for jinja2 on: first fixed version."
2026-11-07T12:42:38 [part4] DONE id="val_06" attempts=2 initial="REVISE" final="PASS" lessons=[] latency_ms=59917.86
2026-11-07T12:43:06 [part4] EVALUATED id="val_07" attempt=1 result="REVISE" explanation="Affected version range is wrong (1.28, 1.31, 1.32)."
2026-11-07T12:43:47 [part4] EVALUATED id="val_07" attempt=2 result="PASS" explanation="Matches the reference for tar on: affected version range."
2026-11-07T12:43:47 [part4] DONE id="val_07" attempts=2 initial="REVISE" final="PASS" lessons=[] latency_ms=68889.99
2026-11-07T12:43:51 [part4] EVALUATED id="val_08" attempt=1 result="PASS" explanation="Matches the reference for protobufjs on: severity."
2026-11-07T12:43:51 [part4] DONE id="val_08" attempts=1 initial="PASS" final="PASS" lessons=[] latency_ms=4380.35
2026-11-07T12:44:12 [part4] EVALUATED id="val_09" attempt=1 result="REVISE" explanation="The package has no reference record and the answer does no
t say so."
2026-11-07T12:44:31 [part4] EVALUATED id="val_09" attempt=2 result="PASS" explanation="Declined correctly: no reference record for this package."
2026-11-07T12:44:31 [part4] DONE id="val_09" attempts=2 initial="REVISE" final="PASS" lessons=[] latency_ms=40300.86
2026-11-07T12:44:35 [part4] EVALUATED id="val_10" attempt=1 result="REVISE" explanation="The package has no reference record and the answer does no
t say so."
2026-11-07T12:45:13 [part4] EVALUATED id="val_10" attempt=2 result="PASS" explanation="Declined correctly: no reference record for this package."
2026-11-07T12:45:13 [part4] DONE id="val_10" attempts=2 initial="REVISE" final="PASS" lessons=[] latency_ms=41588.27
2026-11-07T12:45:13 [part4] VALIDATION with candidate lesson="lesson_001" kept=[]
2026-11-07T12:45:13 [part4] LESSONS LOADED id="val_01" lessons=[["lesson_001", 3]]
2026-11-07T12:45:36 [part4] EVALUATED id="val_01" attempt=1 result="PASS" explanation="Matches the reference for node-fetch on: first fixed version
."
2026-11-07T12:45:36 [part4] DONE id="val_01" attempts=1 initial="PASS" final="PASS" lessons=["lesson_001"] latency_ms=23281.89
2026-11-07T12:45:40 [part4] EVALUATED id="val_02" attempt=1 result="REVISE" explanation="Severity is wrong (critical)."
2026-11-07T12:46:01 [part4] EVALUATED id="val_02" attempt=2 result="PASS" explanation="Matches the reference for ws on: severity."
2026-11-07T12:46:01 [part4] DONE id="val_02" attempts=2 initial="REVISE" final="PASS" lessons=[] latency_ms=24754.43
2026-11-07T12:46:01 [part4] LESSONS LOADED id="val_03" lessons=[["lesson_001", 4]]
2026-11-07T12:46:05 [part4] EVALUATED id="val_03" attempt=1 result="PASS" explanation="Matches the reference for express on: first fixed version."
2026-11-07T12:46:05 [part4] DONE id="val_03" attempts=1 initial="PASS" final="PASS" lessons=["lesson_001"] latency_ms=3879.98
2026-11-07T12:46:08 [part4] EVALUATED id="val_04" attempt=1 result="REVISE" explanation="CVE identifier is wrong (CVE-2021-34542)."
2026-11-07T12:46:12 [part4] EVALUATED id="val_04" attempt=2 result="PASS" explanation="Matches the reference for pillow on: CVE identifier."
2026-11-07T12:46:12 [part4] DONE id="val_04" attempts=2 initial="REVISE" final="PASS" lessons=[] latency_ms=6729.9
```

```
((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 %  make part4-hw09 PHASES="controlled reuse" 2>/dev/null || .venv/bin/python3 -u scripts/
run_hw9_part4.py --phases controlled reuse
2026-11-07T12:50:43 [part4] EVALUATED id="val_06" attempt=1 result="PASS" explanation="Matches the reference for jinja2 on: first fixed version."
2026-11-07T12:50:43 [part4] DONE id="val_06" attempts=1 initial="PASS" final="PASS" lessons=["lesson_001"] latency_ms=39091.88
2026-11-07T12:51:24 [part4] EVALUATED id="val_07" attempt=1 result="REVISE" explanation="Affected version range is wrong (1.32, 1.0, 1.31, 1.32)."
2026-11-07T12:51:55 [part4] EVALUATED id="val_07" attempt=2 result="PASS" explanation="Matches the reference for tar on: affected version range."
2026-11-07T12:51:55 [part4] DONE id="val_07" attempts=2 initial="REVISE" final="PASS" lessons=[] latency_ms=71886.08
2026-11-07T12:51:55 [part4] LESSONS LOADED id="val_08" lessons=[["lesson_002", 3]]
2026-11-07T12:52:11 [part4] EVALUATED id="val_08" attempt=1 result="PASS" explanation="Matches the reference for protobufjs on: severity."
2026-11-07T12:52:11 [part4] DONE id="val_08" attempts=1 initial="PASS" final="PASS" lessons=["lesson_002"] latency_ms=15587.83
2026-11-07T12:52:11 [part4] LESSONS LOADED id="val_09" lessons=[["lesson_001", 6]]
2026-11-07T12:52:14 [part4] EVALUATED id="val_09" attempt=1 result="PASS" explanation="Declined correctly: no reference record for this package."
2026-11-07T12:52:14 [part4] DONE id="val_09" attempts=1 initial="PASS" final="PASS" lessons=["lesson_001"] latency_ms=3164.86
2026-11-07T12:52:14 [part4] LESSONS LOADED id="val_10" lessons=[["lesson_002", 4]]
2026-11-07T12:52:28 [part4] EVALUATED id="val_10" attempt=1 result="REVISE" explanation="The package has no reference record and the answer does no
t say so."
2026-11-07T12:52:43 [part4] EVALUATED id="val_10" attempt=2 result="PASS" explanation="Declined correctly: no reference record for this package."
2026-11-07T12:52:43 [part4] DONE id="val_10" attempts=2 initial="REVISE" final="PASS" lessons=["lesson_002"] latency_ms=29302.97
2026-11-07T12:52:43 [part4] DECISION lesson="lesson_002" keep=true checks={"failures_not_worse": true, "previously_correct_stay_correct": true, "no
_new_unsupported_answers": true}
kept lessons -> data/hw9/lessons.json: ['lesson_001', 'lesson_002']

=== 4.2 Lesson reuse on a later related question ===
2026-11-07T12:52:43 [part4] LESSONS LOADED id="reuse_01" lessons=[["lesson_001", 3]]
2026-11-07T12:52:55 [part4] EVALUATED id="reuse_01" attempt=1 result="PASS" explanation="Matches the reference for ws on: first fixed version."
loaded lesson_001 (created 2026-11-07 from train_05): The answer gives an incorrect first fixed version from memory. -> Before answering a question
 about the first fixed version, look the package up in the s0036_rel advisory database and state the first fixed version exactly as the record give
s it.  use_count 2 -> 3

Question: Which version of ws fixes the header-count denial of service?

[Initial answer - attempt 1]
The first fixed version of `ws` that resolves the header-count denial of service vulnerability (CVE-2024-37890) is 8.17.1. This version is explicit
ly stated in the advisory record VULN-0036-HW9-10 (s0036_rel id 20083) as the fix for the vulnerability affecting versions from 8.0.0 up to but not
 including 8.17.1.
  advisory lookup used: ['VULN-0036-HW9-10 (s0036_rel id 20083): ws affected >= 8.0.0, < 8.17.1, severity high. CVE-2024-37890: Denial of service f
rom a request whose header count exceeds server.maxHeadersCount. Fixed in 8.17.1.']
[Evaluator] result=PASS  explanation: Matches the reference for ws on: first fixed version.

Attempts: 1   Final evaluation: PASS
Final response: The first fixed version of `ws` that resolves the header-count denial of service vulnerability (CVE-2024-37890) is 8.17.1. This ver
sion is explicitly stated in the advisory record VULN-0036-HW9-10 (s0036_rel id 20083) as the fix for the vulnerability affecting versions from 8.0
.0 up to but not including 8.17.1.
```

Figure 15 – 4.3 validation gate: validation runs before and after each lesson, and the keep decisions

```python
def compare(before: list[dict], after: list[dict]) -> dict:
    """The three keep conditions of 4.3, on initial answers (what a lesson changes) and final answers."""

    def ids(rows: list[dict], key: str, value: str) -> set[str]:
        return {r["id"] for r in rows if r[key] == value}

    def unsupported(rows: list[dict]) -> set[str]:
        return {r["id"] for r in rows if any(f["status"] == "unsupported"
                                             for f in r["attempts"][-1]["evaluation"]["failures"])}

    regressed = (ids(before, "initial_result", "PASS") - ids(after, "initial_result", "PASS")) | (
        ids(before, "final_result", "PASS") - ids(after, "final_result", "PASS"))
    checks = {
        "failures_not_worse": len(ids(after, "initial_result", "REVISE")) <= len(ids(before, "initial_result", "REVISE"))
        and len(ids(after, "final_result", "REVISE")) <= len(ids(before, "final_result", "REVISE")),
        "previously_correct_stay_correct": not regressed,
        "no_new_unsupported_answers": unsupported(after) <= unsupported(before),
```

Figure 16 – compare(): the three keep rules of the validation gate (scripts/run_hw9_part4.py)

```
((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 %  make part4-hw09 PHASES="controlled reuse" 2>/dev/null || .venv/bin/python3 -u scripts/run_hw9_part4.py --phases controlled reuse
sion is explicitly stated in the advisory record VULN-0036-HW9-10 (s0036_rel id 20083) as the fix for the vulnerability affecting versions from 8.0.0 up to but not including 8.17.1.

=== 4.4 Final test: four variants on test_data.json ===
2026-11-07T12:52:55 [part4] VARIANT name="Baseline" lessons=[]
2026-11-07T12:53:25 [part4] EVALUATED id="test_01" attempt=1 result="REVISE" explanation="First fixed version is wrong (0.4.20)."
2026-11-07T12:53:25 [part4] DONE id="test_01" attempts=1 initial="REVISE" final="REVISE" lessons=[] latency_ms=30101.6
2026-11-07T12:53:30 [part4] EVALUATED id="test_02" attempt=1 result="REVISE" explanation="Severity is wrong (critical)."
2026-11-07T12:53:30 [part4] DONE id="test_02" attempts=1 initial="REVISE" final="REVISE" lessons=[] latency_ms=4403.94
2026-11-07T12:54:25 [part4] EVALUATED id="test_03" attempt=1 result="REVISE" explanation="First fixed version is wrong (0.0.0, 1.16)."
2026-11-07T12:54:25 [part4] DONE id="test_03" attempts=1 initial="REVISE" final="REVISE" lessons=[] latency_ms=54701.41
2026-11-07T12:54:31 [part4] EVALUATED id="test_04" attempt=1 result="REVISE" explanation="CVE identifier is wrong (CVE-2020-10724)."
2026-11-07T12:54:31 [part4] DONE id="test_04" attempts=1 initial="REVISE" final="REVISE" lessons=[] latency_ms=6616.28
2026-11-07T12:55:17 [part4] EVALUATED id="test_05" attempt=1 result="REVISE" explanation="First fixed version is wrong (1.15.0, 1.15.0, 1.0.0, 1.14.9)."
2026-11-07T12:55:17 [part4] DONE id="test_05" attempts=1 initial="REVISE" final="REVISE" lessons=[] latency_ms=46175.64
2026-11-07T12:55:23 [part4] EVALUATED id="test_06" attempt=1 result="PASS" explanation="Matches the reference for postcss on: severity."
2026-11-07T12:55:23 [part4] DONE id="test_06" attempts=1 initial="PASS" final="PASS" lessons=[] latency_ms=5407.89
2026-11-07T12:55:40 [part4] EVALUATED id="test_07" attempt=1 result="REVISE" explanation="First fixed version is wrong (1.0.3)."
2026-11-07T12:55:40 [part4] DONE id="test_07" attempts=1 initial="REVISE" final="REVISE" lessons=[] latency_ms=17599.63
2026-11-07T12:56:21 [part4] EVALUATED id="test_08" attempt=1 result="REVISE" explanation="Affected version range is wrong (2.22.0, 2.22.1, 2.22.2)."
2026-11-07T12:56:21 [part4] DONE id="test_08" attempts=1 initial="REVISE" final="REVISE" lessons=[] latency_ms=40852.3
2026-11-07T12:56:26 [part4] EVALUATED id="test_09" attempt=1 result="REVISE" explanation="Severity is missing."
2026-11-07T12:56:26 [part4] DONE id="test_09" attempts=1 initial="REVISE" final="REVISE" lessons=[] latency_ms=4876.71
2026-11-07T12:56:43 [part4] EVALUATED id="test_10" attempt=1 result="REVISE" explanation="First fixed version is wrong (1.4.2)."
2026-11-07T12:56:43 [part4] DONE id="test_10" attempts=1 initial="REVISE" final="REVISE" lessons=[] latency_ms=16659.82
2026-11-07T12:57:06 [part4] EVALUATED id="test_11" attempt=1 result="REVISE" explanation="The package has no reference record but the answer states 0.1.0, 1.0.0."
2026-11-07T12:57:06 [part4] DONE id="test_11" attempts=1 initial="REVISE" final="REVISE" lessons=[] latency_ms=23273.95
2026-11-07T12:57:09 [part4] EVALUATED id="test_12" attempt=1 result="PASS" explanation="Declined correctly: no reference record for this package."
2026-11-07T12:57:09 [part4] DONE id="test_12" attempts=1 initial="PASS" final="PASS" lessons=[] latency_ms=3255.09
2026-11-07T12:57:09 [part4] VARIANT name="Evaluator Repair" lessons=[]
2026-11-07T12:57:40 [part4] EVALUATED id="test_01" attempt=1 result="REVISE" explanation="First fixed version is wrong (0.4.20)."
2026-11-07T12:58:21 [part4] EVALUATED id="test_01" attempt=2 result="PASS" explanation="Matches the reference for xml2js on: first fixed version."
2026-11-07T12:58:21 [part4] DONE id="test_01" attempts=2 initial="REVISE" final="PASS" lessons=[] latency_ms=71572.29
2026-11-07T12:58:25 [part4] EVALUATED id="test_02" attempt=1 result="REVISE" explanation="Severity is wrong (critical)."
2026-11-07T12:58:58 [part4] EVALUATED id="test_02" attempt=2 result="PASS" explanation="Matches the reference for urllib3 on: severity."
2026-11-07T12:58:58 [part4] DONE id="test_02" attempts=2 initial="REVISE" final="PASS" lessons=[] latency_ms=37210.03
2026-11-07T12:59:57 [part4] EVALUATED id="test_03" attempt=1 result="REVISE" explanation="First fixed version is wrong (0.0.0, 1.16)."
2026-11-07T13:00:24 [part4] EVALUATED id="test_03" attempt=2 result="PASS" explanation="Matches the reference for golang.org/x/net on: first fixed version."
2026-11-07T13:00:24 [part4] DONE id="test_03" attempts=2 initial="REVISE" final="PASS" lessons=[] latency_ms=85557.4
2026-11-07T13:00:32 [part4] EVALUATED id="test_04" attempt=1 result="REVISE" explanation="CVE identifier is wrong (CVE-2020-10724)."
2026-11-07T13:00:49 [part4] EVALUATED id="test_04" attempt=2 result="PASS" explanation="Matches the reference for moment on: CVE identifier."
2026-11-07T13:00:49 [part4] DONE id="test_04" attempts=2 initial="REVISE" final="PASS" lessons=[] latency_ms=25564.42
2026-11-07T13:01:39 [part4] EVALUATED id="test_05" attempt=1 result="REVISE" explanation="First fixed version is wrong (1.15.0, 1.15.0, 1.0.0, 1.14.9)."
2026-11-07T13:01:56 [part4] EVALUATED id="test_05" attempt=2 result="PASS" explanation="Matches the reference for follow-redirects on: first fixed
```

Figure 17 – 4.2 lesson reuse on a later question (ws, lesson_001, PASS in 1 attempt) and the start of the final test

4.4 Results on the test set

The same 12 questions were used for every variant.

Variant	Correct answers	Incorrect answers	Average attempts	Average latency (ms)
Baseline	2	10	1.00	21,160.4
Evaluator Repair	12	0	1.83	48,639.1
Lesson Memory	12	0	1.17	21,108.6
Validated Improvement	12	0	1.17	19,933.9

The Baseline gets only 2 of 12 right: from memory, the model often gives wrong fixed versions, severities and CVE IDs, and once invented versions for an unknown package. Evaluator Repair fixes all 12 but needs 1.83 attempts and more than twice the latency. With lessons, 10 of 12 first answers are already correct, so the system reaches 12/12 with 1.17 attempts at about Baseline latency. Lesson Memory and Validated Improvement give the same result because validation rejected no lesson, so both variants used the same two lessons.

Agent communication. The two agents pass plain data in-process: the question, then the answer, then the evaluation (result, explanation, feedback), then the revised answer. Lessons are shared through lessons.json.

Stopping condition. The loop stops as soon as the result is PASS. Otherwise it stops after the 2nd attempt and reports whatever the result is. No question ever gets a third attempt.

Limitation. The Evaluator's feedback includes the correct reference value, so a revision almost always passes. In a real system the evaluator wouldn't have better data than the answering agent. The checks are also exact-match rules, so a correct answer phrased differently (for example "2.14.1 and earlier" instead of "< 2.15.0") can be marked wrong.

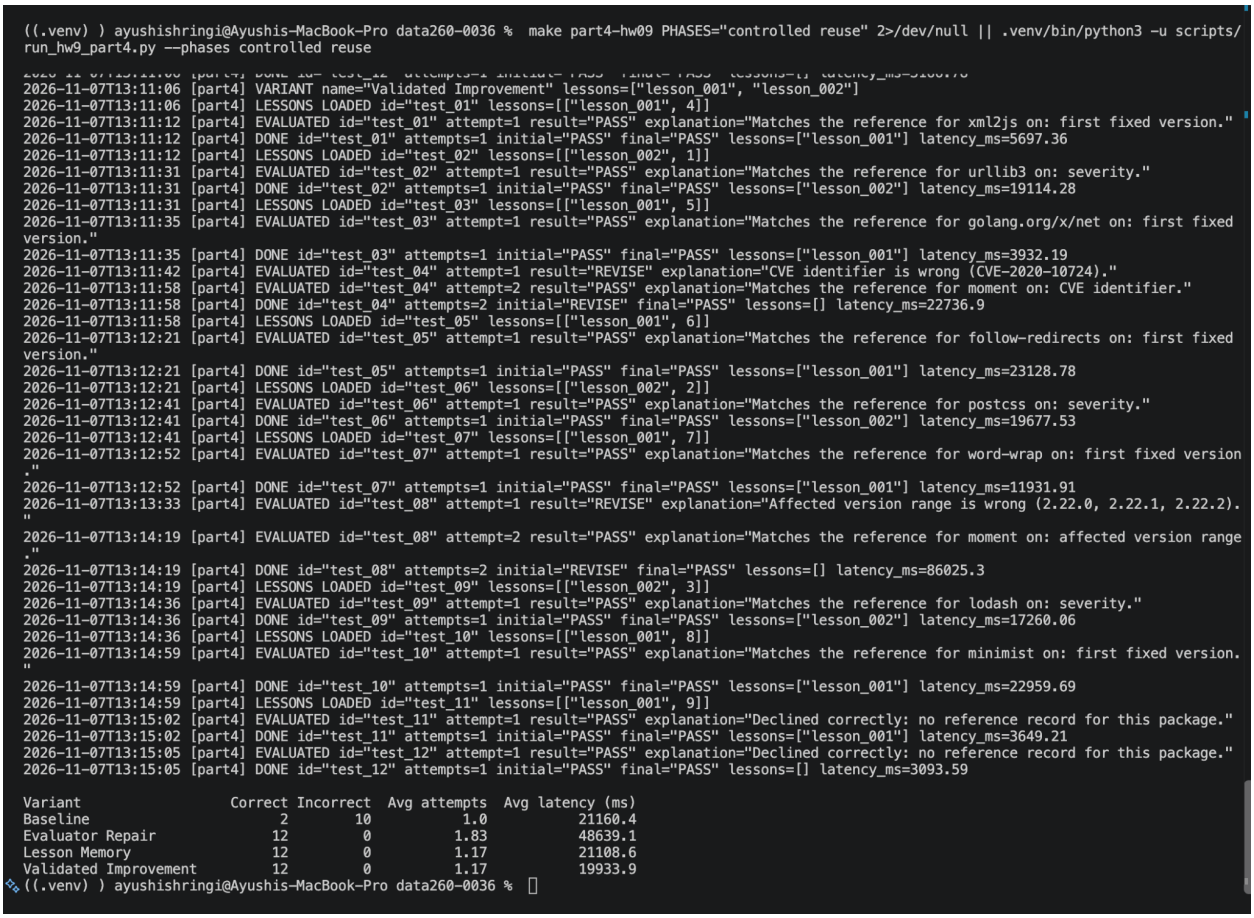


Figure 18 – 4.4 final test: the four variants on test_data.json and the results table

```
                    "latency_ms": round((time.perf_counter() - start) * 1000, 2)}
        system = self.SYSTEM
        if lessons:
            system += "\nLessons from earlier mistakes (follow them):\n" + "\n".join(
                f"- {lesson['correction']}" for lesson in lessons)
        user = f"Question: {question}"
        lookup = None
        if any(lesson.get("action") == "lookup_advisory_db" for lesson in lessons):
            from src.advisory_lookup import fact_line, find_records
            records = find_records(question)
            lookup = [fact_line(r) for r in records]
            user += ("\nAdvisory database (s0036_rel) lookup:\n" + "\n".join(f"- {line}" for lin
                e in lookup)
                     if lookup else "\nAdvisory database (s0036_rel) lookup: no record found for
                          the package "
                                    "in this question.")
        if feedback is not None:
            user += f"\nYour previous answer: {previous}\nEvaluator feedback: {feedback}\nWrite
                a corrected answer."
        text = ask_model(system, user)
        return {"answer": text, "lessons_used": [lesson["lesson_id"] for lesson in lessons],
                "looked_up": lookup is not None, "lookup": lookup, "fault": None,
                "latency_ms": round((time.perf_counter() - start) * 1000, 2)}
```

Figure 19 – Answering Agent: lessons added to the prompt, advisory lookup, and revision with evaluator feedback (src/self_improving.py)

```
202     def evaluate(self, question: str, answer: str) -> dict[str, Any]:
203         """PASS or REVISE with a concise explanation, correction feedback and failure codes."""
204
205         start = time.perf_counter()
206         fact = self.reference.package_in(question)
207         problems: list[str] = []
208         fixes: list[str] = []
209         failures: list[dict[str, str]] = []
210         fields = asked_fields(question)
211         if fact is None:
212             invented = sorted(set(CVE_RE.findall(answer)) | set(VERSION_RE.findall(answer)))
213             if invented:
214                 problems.append(f"the package has no reference record but the answer states {',
                        '.join(invented)}")
215                 failures.append({"pattern": "unknown_package", "status": "unsupported"})
216             elif not DECLINE_RE.search(answer):
217                 problems.append("the package has no reference record and the answer does not say
                         so")
218                 failures.append({"pattern": "unknown_package", "status": "missing"})
219             if failures:
220                 fixes.append("There is no advisory record for this package in the reference data
                        . Say that no "
221                              "advisory record was found and do not state versions, CVE IDs or se
                                 verities.")
222         else:
223             for field in fields:
224                 status, said, expected = self._check(field, fact, answer)
225                 if status == "ok":
226                     continue
227                 failures.append({"pattern": field, "status": status})
228                 problems.append(f"{FIELD_NAMES[field]} {'is wrong (' + said + ')' if status == '
                        wrong' else 'is missing'}")
229                 fixes.append(f"The reference record for {fact['package']} gives the {FIELD_NAMES
                        [field]} as "
230                              f"{expected}; state it exactly.")
231         result = "REVISE" if failures else "PASS"
232         if result == "PASS":
```

Figure 20 – Evaluator Agent: checks the asked fields against the reference and returns PASS/REVISE, explanation and feedback

```
296         attempts: list[dict[str, Any]] = []
297         feedback = previous = None
298         for number in range(1, MAX_ATTEMPTS + 1):
299             answer = self.answering.answer(question, relevant, feedback=feedback, previous=previ
                ous,
300                                            fault=fault if number == 1 else None)
301             evaluation = self.evaluator.evaluate(question, answer["answer"])
302             attempts.append({"attempt": number, **answer, "evaluation": evaluation})
303             self.log("EVALUATED", id=item["id"], attempt=number, result=evaluation["result"],
304                      explanation=evaluation["explanation"])
305             if evaluation["result"] == "PASS" or not self.repair or number == MAX_ATTEMPTS:
306                 break  # stopping condition: PASS, no repair in this variant, or the second atte
                    mpt is done
307             feedback, previous = evaluation["feedback"], answer["answer"]
308         created = self._learn(item, attempts[0]["evaluation"]) if self.learn else []
```

```
315     def _learn(self, item: dict[str, str], evaluation: dict[str, Any]) -> list[str]:
316         """Count failure patterns of the initial answer; create a lesson when one repeats."""
317
318         created = []
319         for failure in evaluation["failures"]:
320             key = failure["pattern"]
321             self.pattern_counts[key] += 1
322             self.pattern_status.setdefault(key, Counter())[failure["status"]] += 1
323             if self.pattern_counts[key] >= self.repeat_threshold and not self.lessons.has_patter
                n(key):
324                 lesson = self.lessons.add({**lesson_for(key, self.pattern_status[key]), "created
                    _from": item["id"]})
325                 created.append(lesson["lesson_id"])
326                 self.log("LESSON CREATED", id=item["id"], lesson=lesson["lesson_id"], pattern=ke
                    y,
327                          seen=self.pattern_counts[key], failure_pattern=lesson["failure_pattern"
                             ])
```

Figure 21 – solve(): the answer → evaluate → revise loop with its stopping condition, and _learn(): a lesson is created when a pattern repeats



Figure 22 – Reference data, training input and lessons.json as used in the run (full files in Appendices D–F)

AI Use

1. What I used AI for and what I did myself. I used an AI coding assistant to help write the Redis view counter, the semantic cache and its query handler, the stale-answer invalidation, the threshold sweep and cost-model scripts, and the Answering/Evaluator agents with the lesson store. I wrote the 40 labeled cases in cache_cases.yaml before the sweep. I ran Redis, Kafka, the agents and every experiment myself, took the screenshots, checked the results against the assignment and wrote the report.

2. One AI output that was wrong. The first version of the cache ran the SentenceTransformers embedding on the Mac GPU (the default device). Cache hits that came right after an agent run took about 150–235 ms instead of a few milliseconds, which made the cached latency look much worse than the cache really is.

3. How I found it. The handler records a timestamp before and after each step. In the Part 2 output, embed_ms was ~200 ms on those hits but ~6 ms on others, while the vector search was ~2 ms every time. The slow embeddings always followed a pipeline run, when Ollama was using the same GPU.

4. What I changed. The embedding model now runs on the CPU by default (HW9_EMBED_DEVICE=cpu), where one query takes about 3 ms whatever the LLM is doing. I reran the Part 2 demo, and the sweep was run with this setting from the start.

Appendix

Appendix A: cache_cases.yaml

```
(.venv) ayushishringi@Ayushis-MacBook-Pro data260-0036 % echo "Name: Ayushi Shringi | SID4 0036"
Name: Ayushi Shringi | SID4 0036
(.venv) ayushishringi@Ayushis-MacBook-Pro data260-0036 % cat cache_cases.yaml
# HW9 Part 3.1 - labeled semantic-cache test set (s0036, DOMAIN_ID 4: open-source package vulner
abilities).
#
# Labels were written BEFORE any threshold sweep and must not be edited after seeing results.
# Labeling mistakes found later are recorded in reports/hw09/LABEL_NOTES.md instead.
#
# How the set is used (scripts/run_hw9_sweep.py):
#   1. The cache is cleared and the 10 seed_queries are answered by the HW8 agents and cached.
#   2. For each threshold, the 40 cases are asked against that same seeded cache
#      (store_on_miss=false, so every threshold sees an identical cache).
#   expected: hit  -> the correct answer is the cached answer of seed `target`
#   expected: miss -> no cached answer is correct (`near` = the seed it superficially resembles)
#   A false hit = a cache hit whose cached answer is not correct for the incoming query.

seed_queries:
  - {id: S01, query: "What is the severity of the Log4Shell vulnerability in log4j-core?"}
  - {id: S02, query: "Which lodash version fixes the template command injection?"}
  - {id: S03, query: "What versions of minimist are affected by the prototype pollution bug?"}
  - {id: S04, query: "How should I fix the PyYAML full_load code execution vulnerability?"}
  - {id: S05, query: "Which requests version stops leaking the Proxy-Authorization header?"}
  - {id: S06, query: "Is Spring4Shell in spring-beans a critical vulnerability?"}
  - {id: S07, query: "What is the CVE for the axios XSRF-TOKEN leak?"}
  - {id: S08, query: "Which jackson-databind version fixes the deeply nested JSON denial of serv
ice?"}
  - {id: S09, query: "What is the impact of the node-fetch cookie leak on redirect?"}
  - {id: S10, query: "Which ws version fixes the header-count denial of service?"}

cases:
  # ---- 10 exact duplicates (should hit) ----
  - {id: C01, category: exact_duplicate, expected: hit, target: S01, query: "What is the severit
y of the Log4Shell vulnerability in log4j-core?"}
  - {id: C02, category: exact_duplicate, expected: hit, target: S02, query: "Which lodash versio
n fixes the template command injection?"}
  - {id: C03, category: exact_duplicate, expected: hit, target: S03, query: "What versions of mi
nimist are affected by the prototype pollution bug?"}
  - {id: C04, category: exact_duplicate, expected: hit, target: S04, query: "How should I fix th
e PyYAML full_load code execution vulnerability?"}
  - {id: C05, category: exact_duplicate, expected: hit, target: S05, query: "Which requests vers
ion stops leaking the Proxy-Authorization header?"}
  - {id: C06, category: exact_duplicate, expected: hit, target: S06, query: "Is Spring4Shell in
spring-beans a critical vulnerability?"}
  - {id: C07, category: exact_duplicate, expected: hit, target: S07, query: "What is the CVE for
 the axios XSRF-TOKEN leak?"}
  - {id: C08, category: exact_duplicate, expected: hit, target: S08, query: "Which jackson-datab
ind version fixes the deeply nested JSON denial of service?"}
  - {id: C09, category: exact_duplicate, expected: hit, target: S09, query: "What is the impact
of the node-fetch cookie leak on redirect?"}
  - {id: C10, category: exact_duplicate, expected: hit, target: S10, query: "Which ws version fi
xes the header-count denial of service?"}

  # ---- 10 paraphrases (should hit: same package, same question, same correct answer) ----
```

Figure A1 – cache_cases.yaml (part 1 of 3)

```
  - {id: C11, category: paraphrase, expected: hit, target: S01, query: "How severe is the log4j-
core Log4Shell issue?"}
  - {id: C12, category: paraphrase, expected: hit, target: S02, query: "What lodash release patc
hes the command injection in template?"}
  - {id: C13, category: paraphrase, expected: hit, target: S03, query: "Which minimist versions
are vulnerable to prototype pollution?"}
  - {id: C14, category: paraphrase, expected: hit, target: S04, query: "How do I remediate the c
ode execution flaw in PyYAML's full_load?"}
  - {id: C15, category: paraphrase, expected: hit, target: S05, query: "What version of requests
 fixes the Proxy-Authorization header leak?"}
  - {id: C16, category: paraphrase, expected: hit, target: S06, query: "How critical is the Spri
ng4Shell vulnerability in spring-beans?"}
  - {id: C17, category: paraphrase, expected: hit, target: S07, query: "Which CVE ID covers the
axios XSRF-TOKEN disclosure?"}
  - {id: C18, category: paraphrase, expected: hit, target: S08, query: "What jackson-databind re
lease patches the DoS from deeply nested JSON?"}
  - {id: C19, category: paraphrase, expected: hit, target: S09, query: "What can an attacker do
with the node-fetch redirect cookie leak?"}
  - {id: C20, category: paraphrase, expected: hit, target: S10, query: "What version of ws patch
es the denial of service from too many headers?"}

  # ---- 10 superficially similar, different correct answer (should miss) ----
  - {id: C21, category: similar_different, expected: miss, near: S01, query: "What is the CVE fo
r the Log4Shell vulnerability in log4j-core?", why: "asks the CVE id (CVE-2021-44228), not the s
everity"}
  - {id: C22, category: similar_different, expected: miss, near: S02, query: "Which pillow versi
on fixes the ImageMath code execution?", why: "different package (pillow 10.2.0, not lodash 4.17
.21)"}
  - {id: C23, category: similar_different, expected: miss, near: S03, query: "What versions of p
rotobufjs are affected by the prototype pollution bug?", why: "different package (protobufjs >=
7.0.0, < 7.2.4)"}
  - {id: C24, category: similar_different, expected: miss, near: S04, query: "What is the severi
ty of the PyYAML full_load code execution vulnerability?", why: "asks the severity (critical), n
ot how to fix it"}
  - {id: C25, category: similar_different, expected: miss, near: S05, query: "Which requests ver
sion first introduced the Proxy-Authorization header leak?", why: "first affected version (2.3.0
), not the fixed version (2.31.0)"}
  - {id: C26, category: similar_different, expected: miss, near: S06, query: "Is the Spring4Shel
l vulnerability in spring-beans 5.3.18 still exploitable?", why: "5.3.18 is the fixed release, s
o the answer is no"}
  - {id: C27, category: similar_different, expected: miss, near: S07, query: "What is the CVE fo
r the express open redirect?", why: "different package (CVE-2024-29041)"}
  - {id: C28, category: similar_different, expected: miss, near: S08, query: "Which django versi
on fixes the regular expression denial of service?", why: "different package (django 4.2.3)"}
  - {id: C29, category: similar_different, expected: miss, near: S09, query: "What is the impact
 of the axios XSRF-TOKEN leak?", why: "different package and token (XSRF token to third-party ho
sts)"}
  - {id: C30, category: similar_different, expected: miss, near: S10, query: "Which ws version i
ntroduced the header-count denial of service?", why: "first affected version (8.0.0), not the fi
x (8.17.1)"}

  # ---- 10 unrelated domain questions (should miss) ----
  - {id: C31, category: unrelated, expected: miss, query: "How do I generate an SBOM for a Pytho
n project?"}
```

Figure A2 – cache_cases.yaml (part 2 of 3)

```
  - {id: C32, category: unrelated, expected: miss, query: "What is the difference between CVSS b
ase and temporal scores?"}
  - {id: C33, category: unrelated, expected: miss, query: "How often should we run npm audit in
CI?"}
  - {id: C34, category: unrelated, expected: miss, query: "What does a CWE identifier represent?
"}
  - {id: C35, category: unrelated, expected: miss, query: "How do I pin transitive dependencies
with pip-tools?"}
  - {id: C36, category: unrelated, expected: miss, query: "What is a GitHub Security Advisory (G
HSA) ID?"}
  - {id: C37, category: unrelated, expected: miss, query: "Should a reporter disclose a vulnerab
ility publicly before a fix exists?"}
  - {id: C38, category: unrelated, expected: miss, query: "How does Dependabot decide when to op
en a pull request?"}
  - {id: C39, category: unrelated, expected: miss, query: "What is typosquatting in package regi
stries?"}
  - {id: C40, category: unrelated, expected: miss, query: "How do I verify the signature of a Ma
ven artifact?"}
(.venv) ayushishringi@Ayushis-MacBook-Pro data260-0036 %
```

Figure A3 – cache_cases.yaml (part 3 of 3)

Appendix B: validation_data.json

```
(.venv) ayushishringi@Ayushis-MacBook-Pro data260-0036 % echo "Name: Ayushi Shringi |
 SID4 0036"
Name: Ayushi Shringi | SID4 0036
(.venv) ayushishringi@Ayushis-MacBook-Pro data260-0036 % cat data/hw9/validation_data
.json
[
  {
    "id": "val_01",
    "question": "Which version of node-fetch fixes the cookie leak on redirect?"
  },
  {
    "id": "val_02",
    "question": "What is the severity of the ws header-count denial of service?"
  },
  {
    "id": "val_03",
    "question": "Which express version fixes the open redirect vulnerability?"
  },
  {
    "id": "val_04",
    "question": "What is the CVE ID for the pillow ImageMath.eval code execution?"
  },
  {
    "id": "val_05",
    "question": "How severe is the django EmailValidator ReDoS?"
  },
  {
    "id": "val_06",
    "question": "Which version of jinja2 patches the xmlattr XSS?"
  },
  {
    "id": "val_07",
    "question": "Which versions of tar are affected by the symlink file overwrite?"
  },
  {
    "id": "val_08",
    "question": "What is the severity of the protobufjs prototype pollution?"
  },
  {
    "id": "val_09",
    "question": "Which version of yamlparse-lite fixes its code injection bug?"
  },
  {
    "id": "val_10",
    "question": "What is the severity of the netconfig-js command injection?"
  }
]
(.venv) ayushishringi@Ayushis-MacBook-Pro data260-0036 %
```

Figure B1 – validation_data.json (10 questions)

Appendix C: test_data.json

```
(.venv) ayushishringi@Ayushis-MacBook-Pro data260-0036 % echo "Name: Ayushi Shringi |
 SID4 0036"
Name: Ayushi Shringi | SID4 0036
(.venv) ayushishringi@Ayushis-MacBook-Pro data260-0036 % cat data/hw9/test_data.json
[
  {
    "id": "test_01",
    "question": "Which version of xml2js fixes the __proto__ prototype pollution?"
  },
  {
    "id": "test_02",
    "question": "What is the severity of the urllib3 303 redirect body leak?"
  },
  {
    "id": "test_03",
    "question": "Which golang.org/x/net version fixes the HTTP/2 rapid reset attack?"
  },
  {
    "id": "test_04",
    "question": "What is the CVE ID for the moment RFC 2822 ReDoS?"
  },
  {
    "id": "test_05",
    "question": "Which version of follow-redirects stops keeping the Proxy-Authorizat
ion header across hosts?"
  },
  {
    "id": "test_06",
    "question": "How severe is the postcss line-return parsing issue?"
  },
  {
    "id": "test_07",
    "question": "Which word-wrap version fixes the ReDoS in wrap?"
  },
  {
    "id": "test_08",
    "question": "Which versions of moment are affected by the RFC 2822 parsing ReDoS?
"
  },
  {
    "id": "test_09",
    "question": "What is the severity of the lodash template command injection?"
  },
  {
    "id": "test_10",
    "question": "Which version of minimist fixes the prototype pollution?"
  },
  {
    "id": "test_11",
    "question": "Which version of pyquickhash fixes its buffer overflow?"
  },
  {
    "id": "test_12",
    "question": "What is the CVE ID for the tiny-xmlrpc deserialization flaw?"
  }
]
(.venv) ayushishringi@Ayushis-MacBook-Pro data260-0036 %
```

Figure C1 – test_data.json (12 questions)

Appendix D: training_data.json

```
(.venv) ayushishringi@Ayushis-MacBook-Pro data260-0036 % echo "Name: Ayushi Shringi |
 SID4 0036"
Name: Ayushi Shringi | SID4 0036
(.venv) ayushishringi@Ayushis-MacBook-Pro data260-0036 % cat data/hw9/training_data.j
son
[
  {
    "id": "train_01",
    "question": "Which version of lodash fixes CVE-2021-23337?"
  },
  {
    "id": "train_02",
    "question": "What is the severity of the minimist prototype pollution vulnerabili
ty?"
  },
  {
    "id": "train_03",
    "question": "Which pyyaml version first fixed the full_load code execution issue?
"
  },
  {
    "id": "train_04",
    "question": "What is the CVE ID for the Spring4Shell vulnerability in spring-bean
s?"
  },
  {
    "id": "train_05",
    "question": "Which requests version fixes the Proxy-Authorization header leak?"
  },
  {
    "id": "train_06",
    "question": "How severe is the axios XSRF-TOKEN disclosure?"
  },
  {
    "id": "train_07",
    "question": "Which versions of jackson-databind are affected by the nested JSON d
enial of service?"
  },
  {
    "id": "train_08",
    "question": "Which version of log4j-core first fixed Log4Shell?"
  },
  {
    "id": "train_09",
    "question": "What version of fastjsonx fixes its deserialization vulnerability?"
  },
  {
    "id": "train_10",
    "question": "What is the CVE ID for the leftpad-plus prototype pollution bug?"
  }
]
(.venv) ayushishringi@Ayushis-MacBook-Pro data260-0036 %
```

Figure D1 – training_data.json (10 questions)

Appendix E: Reference dataset

```
(.venv) ayushishringi@Ayushis-MacBook-Pro data260-0036 % echo "Name: Ayushi Shringi | SID4 0036"
Name: Ayushi Shringi | SID4 0036
(.venv) ayushishringi@Ayushis-MacBook-Pro data260-0036 % jq -r '.description' data/hw9/reference_facts.json
HW9 reference facts (ground truth) for open-source package vulnerabilities, compiled from public advisories (NVD / GitHub
 Security Advisories). The same rows are seeded into s0036_rel.vulnerability_reports by scripts/migrate_hw9.py. Severity
uses the four DOMAIN_SCHEMA buckets; GitHub 'moderate' is stored as medium.
(.venv) ayushishringi@Ayushis-MacBook-Pro data260-0036 % jq -c '.facts[]' data/hw9/reference_facts.json
{"package":"log4j-core","cve":"CVE-2021-44228","severity":"critical","affected_versions":">= 2.0-beta9, < 2.15.0","fixed_
version":"2.15.0","summary":"Remote code execution through JNDI lookups in logged strings (Log4Shell).","report_code":"VU
LN-0036-HW9-01"}
{"package":"lodash","cve":"CVE-2021-23337","severity":"high","affected_versions":"< 4.17.21","fixed_version":"4.17.21","s
ummary":"Command injection through the template function.","report_code":"VULN-0036-HW9-02"}
{"package":"minimist","cve":"CVE-2021-44906","severity":"critical","affected_versions":">= 1.0.0, < 1.2.6","fixed_version
":"1.2.6","summary":"Prototype pollution through crafted command-line arguments.","report_code":"VULN-0036-HW9-03"}
{"package":"pyyaml","cve":"CVE-2020-14343","severity":"critical","affected_versions":"< 5.4","fixed_version":"5.4","summa
ry":"Arbitrary code execution when untrusted YAML is parsed with full_load or FullLoader.","report_code":"VULN-0036-HW9-0
4"}
{"package":"requests","cve":"CVE-2023-32681","severity":"medium","affected_versions":">= 2.3.0, < 2.31.0","fixed_version"
:"2.31.0","summary":"Proxy-Authorization header leaked to the destination server on HTTPS redirects.","report_code":"VULN
-0036-HW9-05"}
{"package":"spring-beans","cve":"CVE-2022-22965","severity":"critical","affected_versions":">= 5.3.0, < 5.3.18","fixed_ve
rsion":"5.3.18","summary":"Remote code execution through data binding on JDK 9+ (Spring4Shell).","report_code":"VULN-0036
-HW9-06"}
{"package":"axios","cve":"CVE-2023-45857","severity":"medium","affected_versions":">= 0.8.1, < 1.6.0","fixed_version":"1.
6.0","summary":"XSRF-TOKEN cookie value disclosed to third-party hosts.","report_code":"VULN-0036-HW9-07"}
{"package":"jackson-databind","cve":"CVE-2020-36518","severity":"high","affected_versions":">= 2.13.0, < 2.13.2.1","fixed
_version":"2.13.2.1","summary":"Denial of service through deeply nested JSON (stack overflow).","report_code":"VULN-0036-
HW9-08"}
{"package":"node-fetch","cve":"CVE-2022-0235","severity":"high","affected_versions":"< 2.6.7","fixed_version":"2.6.7","su
mmary":"Cookie and Authorization headers forwarded to a third-party host on redirect.","report_code":"VULN-0036-HW9-09"}
{"package":"ws","cve":"CVE-2024-37890","severity":"high","affected_versions":">= 8.0.0, < 8.17.1","fixed_version":"8.17.1
","summary":"Denial of service from a request whose header count exceeds server.maxHeadersCount.","report_code":"VULN-003
6-HW9-10"}
{"package":"express","cve":"CVE-2024-29041","severity":"medium","affected_versions":"< 4.19.2","fixed_version":"4.19.2","
summary":"Open redirect through malformed URLs passed to res.location and res.redirect.","report_code":"VULN-0036-HW9-11"
}
{"package":"pillow","cve":"CVE-2023-50447","severity":"high","affected_versions":"< 10.2.0","fixed_version":"10.2.0","sum
mary":"Arbitrary code execution through the environment parameter of ImageMath.eval.","report_code":"VULN-0036-HW9-12"}
{"package":"django","cve":"CVE-2023-36053","severity":"high","affected_versions":">= 4.2, < 4.2.3","fixed_version":"4.2.3
","summary":"Regular expression denial of service in EmailValidator and URLValidator.","report_code":"VULN-0036-HW9-13"}
{"package":"jinja2","cve":"CVE-2024-22195","severity":"medium","affected_versions":"< 3.1.3","fixed_version":"3.1.3","sum
mary":"Cross-site scripting through the xmlattr filter.","report_code":"VULN-0036-HW9-14"}
{"package":"tar","cve":"CVE-2021-32803","severity":"high","affected_versions":">= 6.0.0, < 6.1.2","fixed_version":"6.1.2"
,"summary":"Arbitrary file creation or overwrite through symbolic-link poisoning.","report_code":"VULN-0036-HW9-15"}
{"package":"protobufjs","cve":"CVE-2023-36665","severity":"critical","affected_versions":">= 7.0.0, < 7.2.4","fixed_versi
on":"7.2.4","summary":"Prototype pollution through crafted protobuf definitions.","report_code":"VULN-0036-HW9-16"}
{"package":"xml2js","cve":"CVE-2023-0842","severity":"medium","affected_versions":"< 0.5.0","fixed_version":"0.5.0","summ
ary":"Prototype pollution through the __proto__ key in parsed XML.","report_code":"VULN-0036-HW9-17"}
{"package":"urllib3","cve":"CVE-2023-45803","severity":"medium","affected_versions":"< 1.26.18","fixed_version":"1.26.18"
,"summary":"Request body not removed when a 303 redirect changes the method to GET.","report_code":"VULN-0036-HW9-18"}
{"package":"golang.org/x/net","cve":"CVE-2023-44487","severity":"high","affected_versions":"< 0.17.0","fixed_version":"0.
17.0","summary":"HTTP/2 rapid reset denial of service.","report_code":"VULN-0036-HW9-19"}
{"package":"moment","cve":"CVE-2022-31129","severity":"high","affected_versions":">= 2.18.0, < 2.29.4","fixed_version":"2
.29.4","summary":"Regular expression denial of service when parsing RFC 2822 dates.","report_code":"VULN-0036-HW9-20"}
{"package":"follow-redirects","cve":"CVE-2024-28849","severity":"medium","affected_versions":"< 1.15.6","fixed_version":"
1.15.6","summary":"Proxy-Authorization header kept across hosts on redirect.","report_code":"VULN-0036-HW9-21"}
{"package":"postcss","cve":"CVE-2023-44270","severity":"medium","affected_versions":"< 8.4.31","fixed_version":"8.4.31","
summary":"Line-return parsing error that lets crafted CSS inject comments.","report_code":"VULN-0036-HW9-22"}
{"package":"word-wrap","cve":"CVE-2023-26115","severity":"medium","affected_versions":"< 1.2.4","fixed_version":"1.2.4","
summary":"Regular expression denial of service in the wrap function.","report_code":"VULN-0036-HW9-23"}
(.venv) ayushishringi@Ayushis-MacBook-Pro data260-0036 %
```

Figure E1 – reference_facts.json: the reference dataset (23 packages, one fact per line)

Appendix F: lessons.json

```
(.venv) ayushishringi@Ayushis-MacBook-Pro data260-0036 % echo "Name: Ayushi Shringi | SID4 0036"
Name: Ayushi Shringi | SID4 0036
(.venv) ayushishringi@Ayushis-MacBook-Pro data260-0036 % cat data/hw9/lessons.json
[
  {
    "lesson_id": "lesson_001",
    "failure_pattern": "The answer gives an incorrect first fixed version from memory.",
    "correction": "Before answering a question about the first fixed version, look the package up in the s0036_r
el advisory database and state the first fixed version exactly as the record gives it.",
    "created_at": "2026-11-07",
    "use_count": 9,
    "pattern_key": "fixed_version",
    "applies_to": [
      "fixed_version"
    ],
    "action": "lookup_advisory_db",
    "created_from": "train_05"
  },
  {
    "lesson_id": "lesson_002",
    "failure_pattern": "The answer gives an incorrect severity from memory.",
    "correction": "Before answering a question about the severity, look the package up in the s0036_rel advisory
 database and state the severity exactly as the record gives it.",
    "created_at": "2026-11-07",
    "use_count": 3,
    "pattern_key": "severity",
    "applies_to": [
      "severity"
    ],
    "action": "lookup_advisory_db",
    "created_from": "train_06"
  }
]
(.venv) ayushishringi@Ayushis-MacBook-Pro data260-0036 %
```

Figure F1 – lessons.json after all runs (use_count includes the validation and test runs)